

String

19 July 2026 09:04

Why is String immutable ?

Why should I use StringBuilder ?

Why this topic is important .

Real example:

1. Username
2. Password
3. Email
4. Mobile number
5. JWT token
6. API url
7. SQL query
8. JSON message
9. File path
10. Request parameter

Note: **StringBuilder** and **StringBuffer** are important when text changes many times.

Cloneable is important when we want to copy of an object instead of sharing the same object.

What is **String** ?

String is class in java used to store text.

String is a sequence of character:



Ways of creating String object

1. String Literal

String s1="java"

2. Using new keyword

```
String s2= new String("Java");
```

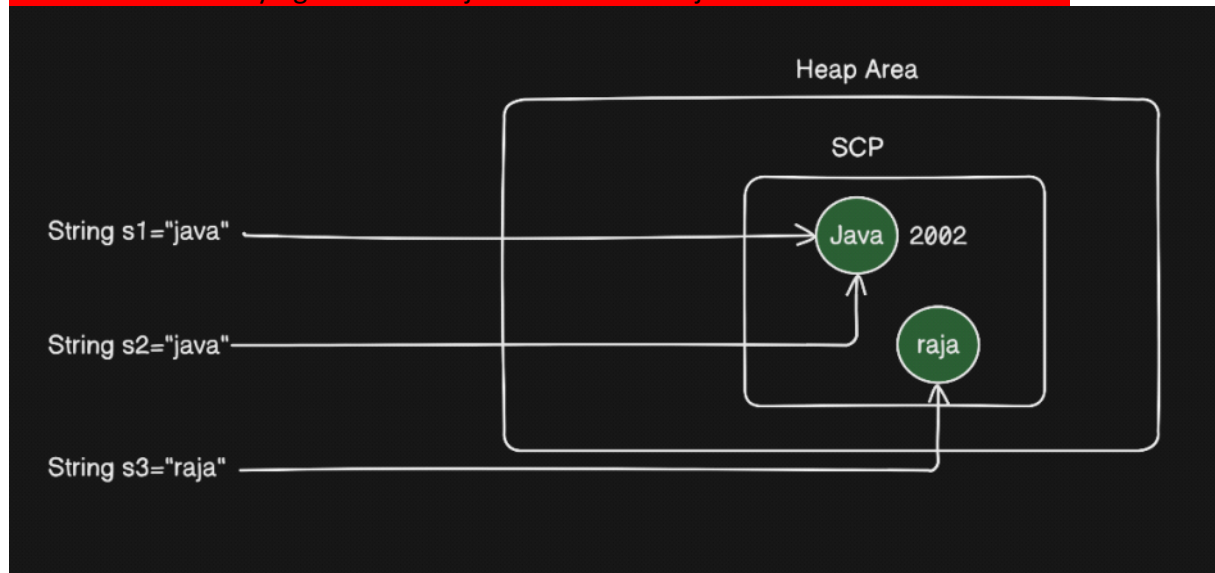
String Constant pool / SCP

SCP is a special memory area where java store String literals

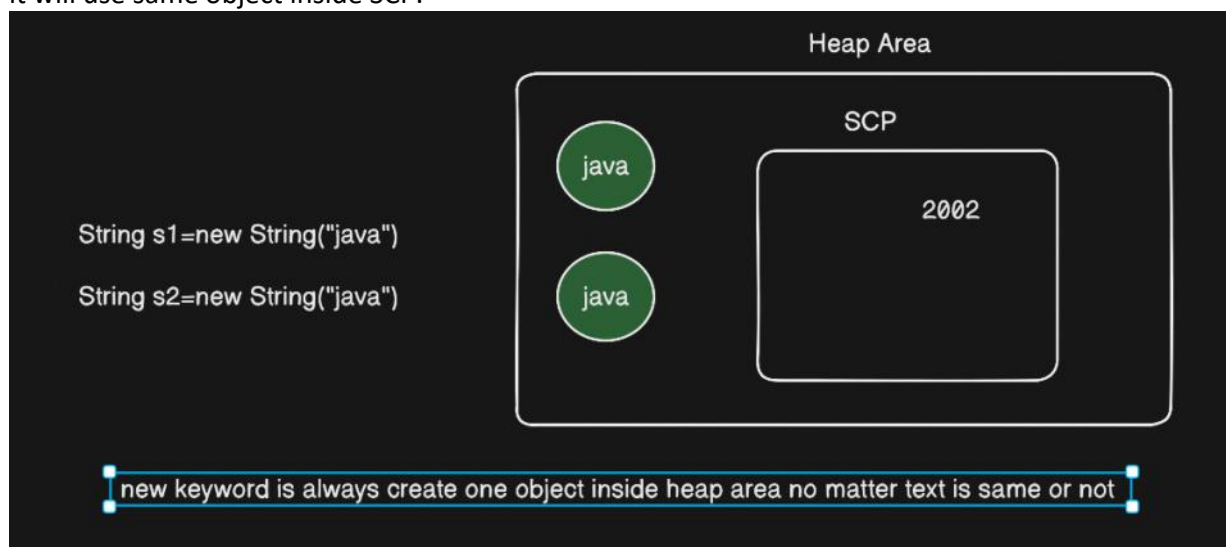
Why java use SCP ?

To reuse the same text and save memory

Whenever we are trying to create object with Literal object will be created inside SCP.



When we are trying to create object with same text available inside SCP then JVM will not create new object for us it will use same object inside SCP.



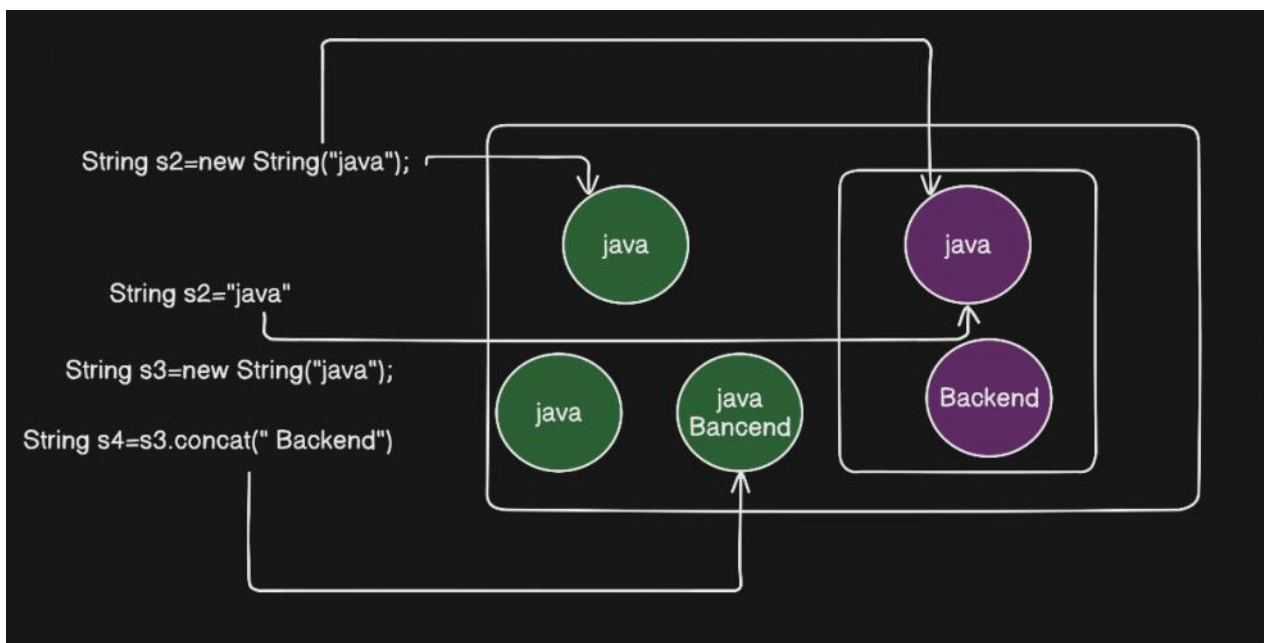
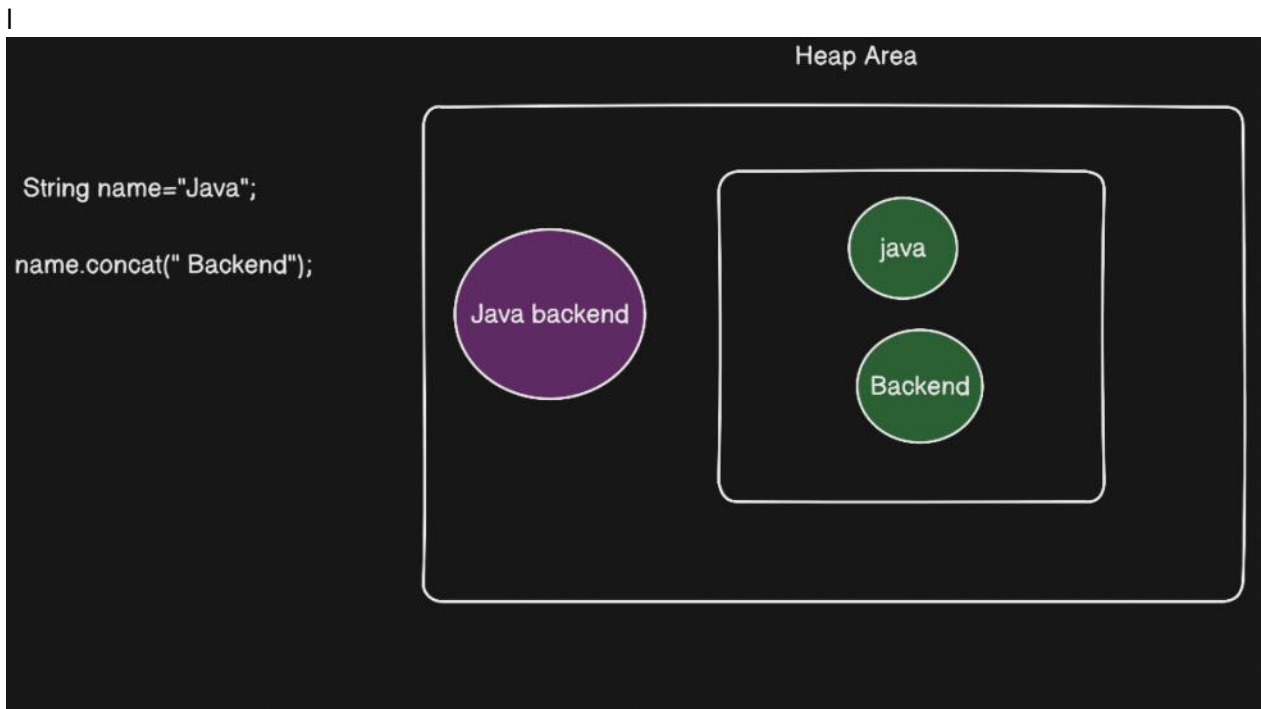
String is Immutable

Immutable means cannot be changed.

|

Note: once a String object is created, it's content cannot be modified.

|



Why String is Immutable ?

String is immutable for important reasons.

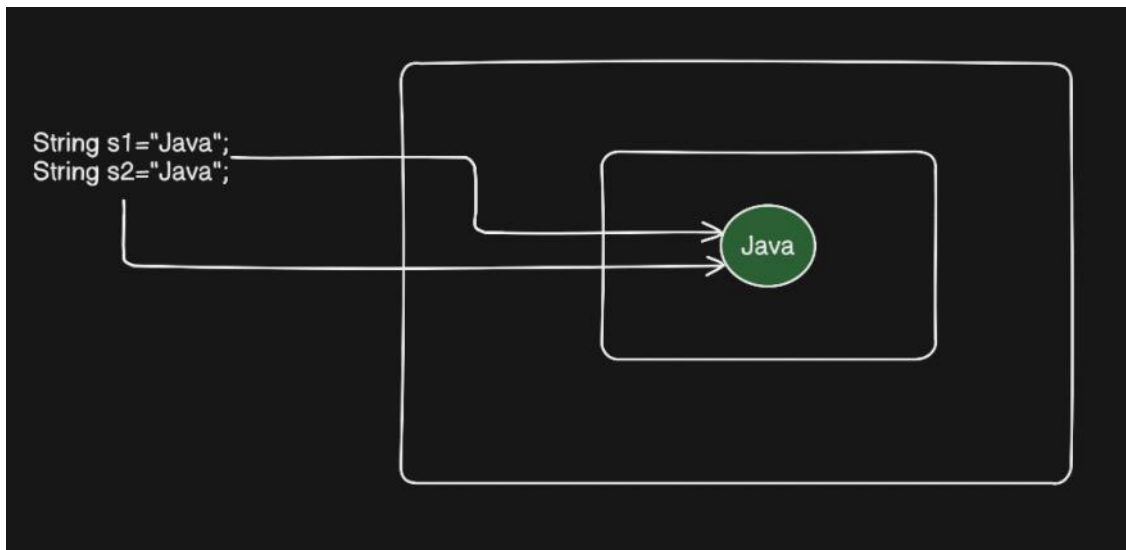
Reason 1: String constant pool safety

Eg:

String s1="java"

String s2="java"

Both may share same object in SCP



If String were mutable :

1. Changing s1 would also affect s2 that would be dangerous.

Reason 2 : Security

String are used in:

1. File path
2. Database url
3. Username
4. Password
5. Network connection
6. Class loading

If string could change unexpectedly, security issue could happen

Reason 3: HashMap key safety

String are commonly used as HashMap keys

Reason 4: Thread safety

Immutable objects are naturally safer to share between threads.

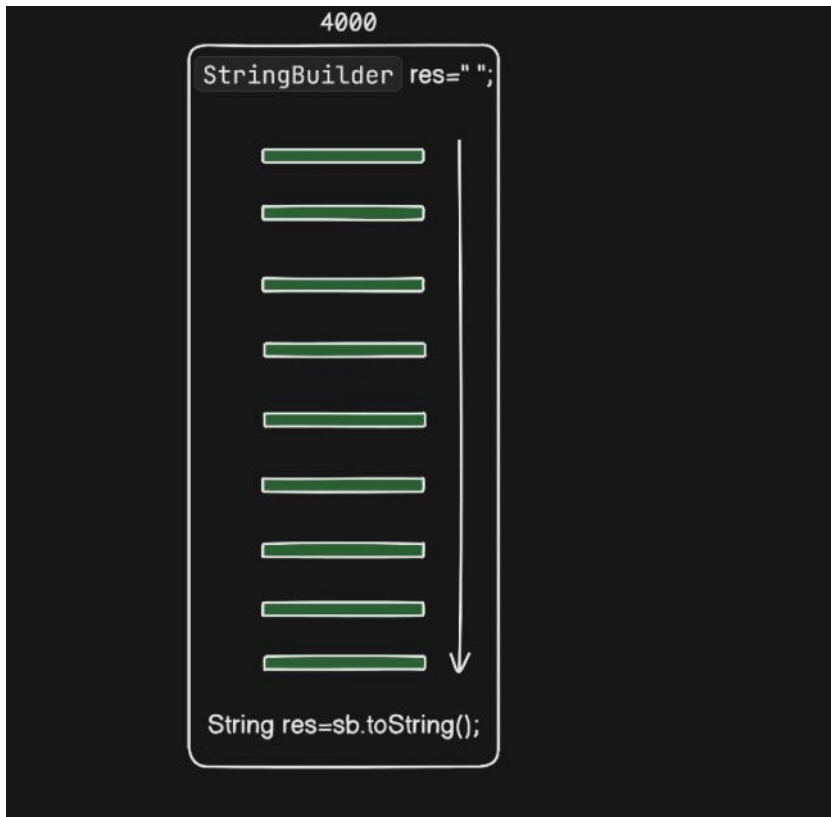
Notes: String immutability gives memory saving, security, stable hashing and thread safety.

StringBuilder

StringBuilder is a mutable class used to modify text.

StringBuilder internally uses a character array-like storage.

append()
insert()
delete()
replace()
reverse()
capacity()
length()
toString()



StringBuffer

StringBuffer is also mutable like StringBuilder but StringBuffer is Thread safe (synchronized)

What does thread-safe mean here ?

Multiple thread can use same object without corrupting internal data.

StringBuffer methods are synchronised.

Conceptually

```
@Override
@IntrinsicCandidate
public synchronized StringBuffer append(String str) {
    toStringCache = null;
    super.append(str);
    return this;
}
```

This means:

Only one thread can append at a time on same StringBuffer Object.

Real use case:

If multiple threads are modifying same text object, StringBuffer is safe

String Vs StringBuilder Vs StringBuffer

Points	String	StringBuilder	StringBuffer
Mutable ?	NO	Yes	Yes
Thread-safe	Yes because immutable	No	Yes
Performance	Slow	Fast	Slower then StringBuilder
Storage idea	Immutable object	Growable character	Growable synchronizes character
Best use	Fixed text	Repeated changes	Repeated changes in multi-threads

Real Time Selection

Situation	Use
username	String
Email	String
Build SQL query	StringBuilder
Build large CSV output	StringBuilder
Shared log text modified by multiple thread	StringBuffer

What is Cloneable ?

Now will understand how to copy object

Problem: Sometime we do not want to reference pointing to same object., we want separate copy.

Class Student

```
{
    int id;
    String name;
}
```

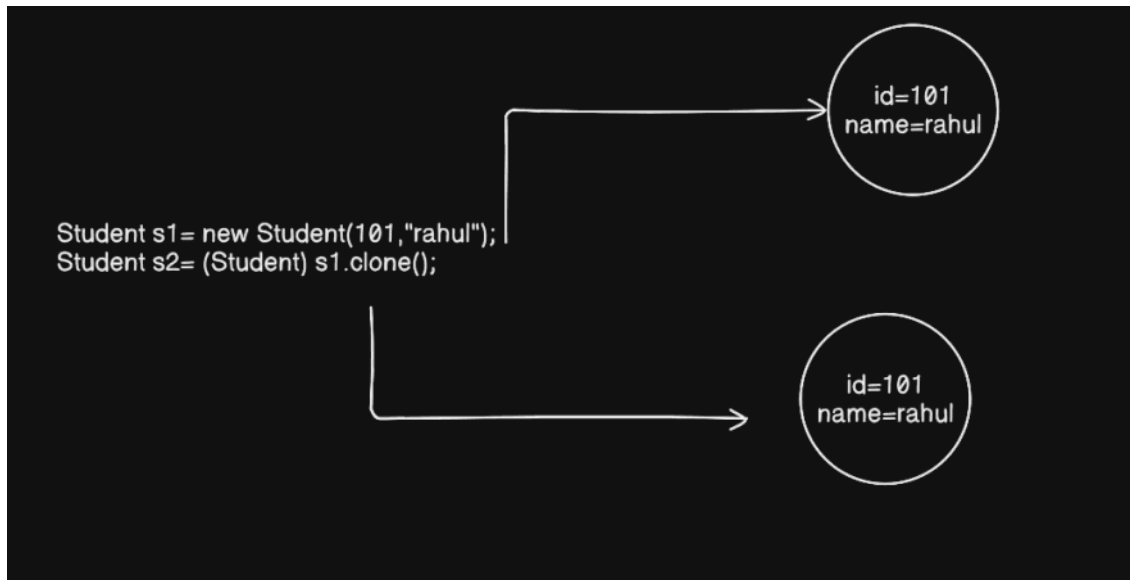
```
Student s1=new Student();
```

```
Student s2=s1;
```



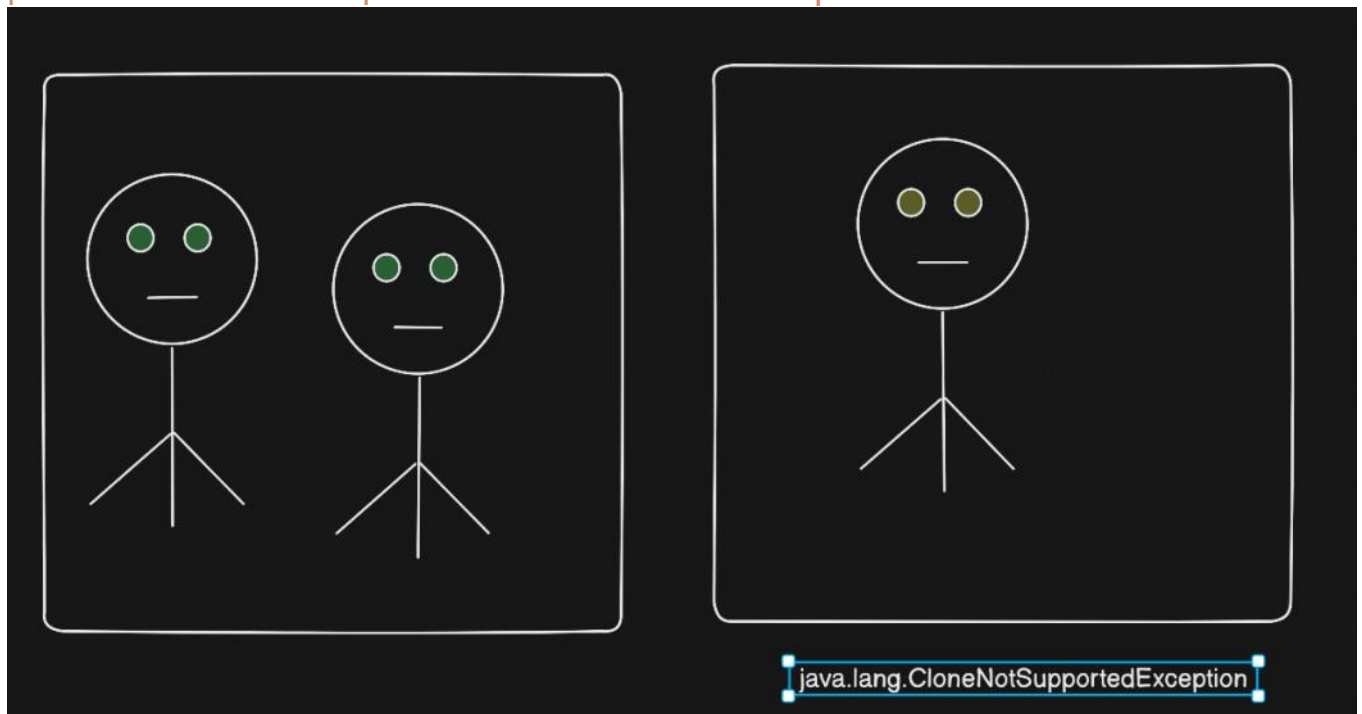
Cloneable interface:

Cloneable is **marker** interface. This interface with no method, it gives information/permission to JVM



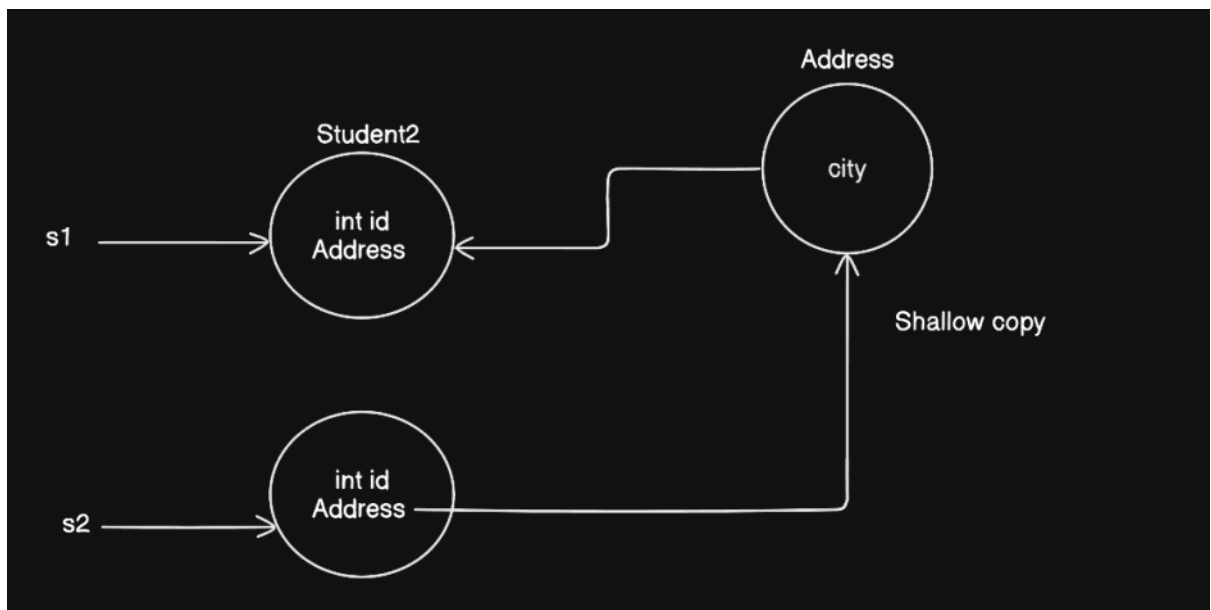
public class Student implements Cloneable

public class Student

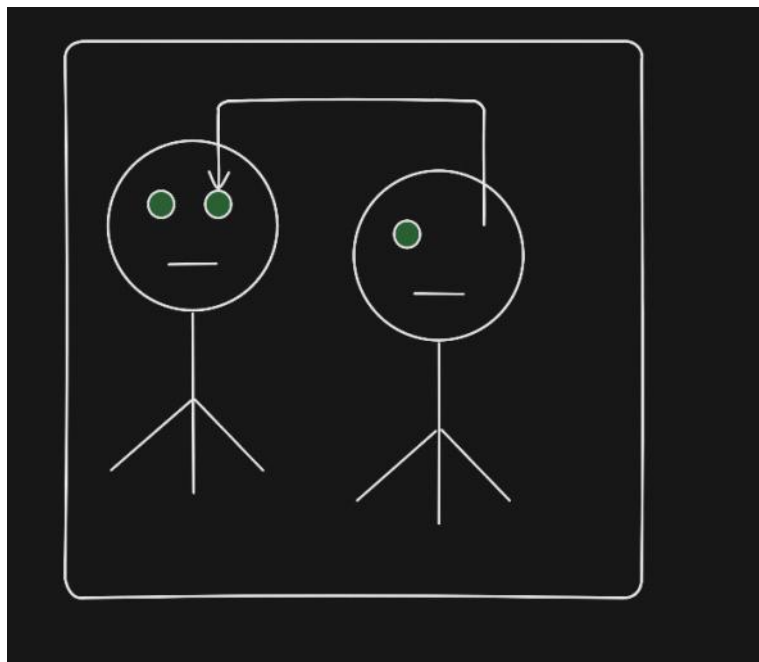


Shallow Copy And Deep Copy





s1 and s2 are two different object but both share same address object.



Deep Copy:

Create new object for referenced object also